

بسم الله الرحمن الرحيم

گزارش کار آزمایشگاه سیستم عامل

پروژه: سیستم مدیریت اطلاعات دانشجویان با Shell Script

نام درس: آزمایشگاه سیستم عامل

نام استاد: سرکار خانم صدیقی وحیدی

نام دانشجو: عطیه جعفرزاده رضانی

شماره دانشجویی: 402103206

1. مقدمه و هدف پروژه

در این پروژه که مربوط به آزمایشگاه درس سیستم عامل می باشد، هدف پیاده سازی یک برنامه ی مدیریتی ساده جهت ثبت اطلاعات دانشجویان، ورود نمرات و گزارش گیری با استفاده از دستورات **Shell** در محیط سیستم عامل **Linux** است.

این برنامه به صورت کامل با **Shell Script (Bash)** نوشته شده و برای ایجاد رابط کاربری، از ابزار **dialog** استفاده شده است.

تمامی ورودی ها و خروجی ها از طریق پنجره های دیالوگی انجام می شود و هیچ ورودی متنی مستقیم از ترمینال دریافت نمی شود.

اطلاعات دانشجویان و نمرات آن ها در فایل ذخیره شده و امکان گزارش گیری های مختلف فراهم شده است.

2. ابزارها و محیط اجرا

- سیستم عامل **Ubuntu Linux**
 - زبان برنامه نویسی **Bash (Shell Script)**
 - ابزار رابط کاربری **dialog**
 - فایل های داده:
- **students.db** (اطلاعات دانشجویان)
 - **grades.db** (نمرات دانشجویان)

3. اجرای برنامه

برنامه با دستور زیر اجرا می‌شود:

```
chmod +x project.sh
./project.sh
```

منوی اصلی برنامه

منوی اصلی با استفاده از دستور `dialog --menu` پیاده‌سازی شده و شامل سه گزینه است:

- Register Student
- Enter Grades
- Reports

```
main_menu() {
  while true; do
    dialog --clear --title "Operating Systems Project" \
      --menu "Main Menu:" 12 55 0 \
        1 "Register Student" \
        2 "Enter Grades" \
        3 "Reports" \
        2> "$TMP"

    if [ $? -ne 0 ]; then
      clear
      exit 0
    fi

    case "$(cat "$TMP")" in
      1) register_students ;;
      2) enter_grades ;;
      3) reports_menu ;;
    esac
  done
}
```

تعریف تابع

```
main_menu() {
```

این تابع منوی اصلی برنامه را نمایش می‌دهد و نقطه‌ی مرکزی اجرای کل پروژه است. تمام بخش‌ها (ثبت دانشجو، ورود نمرات، گزارش‌گیری) از این منو فراخوانی می‌شوند.

حلقه‌ی اصلی برنامه

```
while true; do
```

برای اینکه بعد از انجام هر عملیات دوباره به منوی اصلی بازگردیم، از یک حلقه‌ی بی‌نهایت استفاده شده است. با این کار برنامه تا زمانی که کاربر خارج نشود، فعال می‌ماند.

نمایش منوی اصلی با dialog

```
dialog --clear --title "Operating Systems Project" \
```

- صفحه‌ی قبلی پاک می‌شود(--clear)
- عنوان پنجره **Operating Systems Project** :

```
--menu "Main Menu:" 12 55 0 \
```

یک منوی دیالوگی ساخته می‌شود با عنوان **Main Menu**
اندازه پنجره: ارتفاع 12 و عرض 55

گزینه‌های منوی اصلی

```
1 "Register Student" \  
2 "Enter Grades" \  
3 "Reports" \
```

سه گزینه مطابق صورت‌سؤال نمایش داده می‌شود:

1. ثبت دانشجو
2. ورود نمرات
3. گزارش‌گیری

ذخیره انتخاب کاربر

```
2> "$TMP"
```

انتخاب کاربر در فایل موقت \$TMP ذخیره می‌شود تا در ادامه با case پردازش شود.

خروج از برنامه با Cancel / ESC

```
if [ $? -ne 0 ]; then
    clear
    exit 0
fi
```

اگر کاربر در منوی اصلی دکمه Cancel یا ESC را بزند:

- صفحه ترمینال پاک می‌شود (clear)
- برنامه به طور کامل بسته می‌شود (exit 0)

بررسی انتخاب کاربر (case)

```
case "$(cat "$TMP")" in
```

مقدار انتخاب‌شده از \$TMP خوانده می‌شود و با case بررسی می‌گردد.

انتخاب گزینه 1: ثبت دانشجو

```
1) register_students ;;
```

اگر کاربر گزینه 1 را انتخاب کند، تابع register_students اجرا می‌شود و فرم ثبت دانشجو نمایش داده می‌شود.

انتخاب گزینه 2: ورود نمرات

```
2) enter_grades ;;
```

اگر گزینه 2 انتخاب شود، تابع enter_grades اجرا می‌شود و فرم ورود نمرات نمایش داده می‌شود.

انتخاب گزینه 3: گزارش‌گیری

```
3) reports_menu ;;
```

اگر گزینه 3 انتخاب شود، تابع reports_menu اجرا می‌شود و منوی گزارش‌گیری نمایش داده می‌شود.

پایان case و تکرار منو

```
esac
```

پس از پایان اجرای هر بخش، برنامه به حلقه برمی‌گردد و منوی اصلی دوباره نمایش داده می‌شود. این دقیقاً همان رفتاری است که در صورت سوال خواسته شده است (بازگشت به منوی اصلی پس از پایان هر بخش).

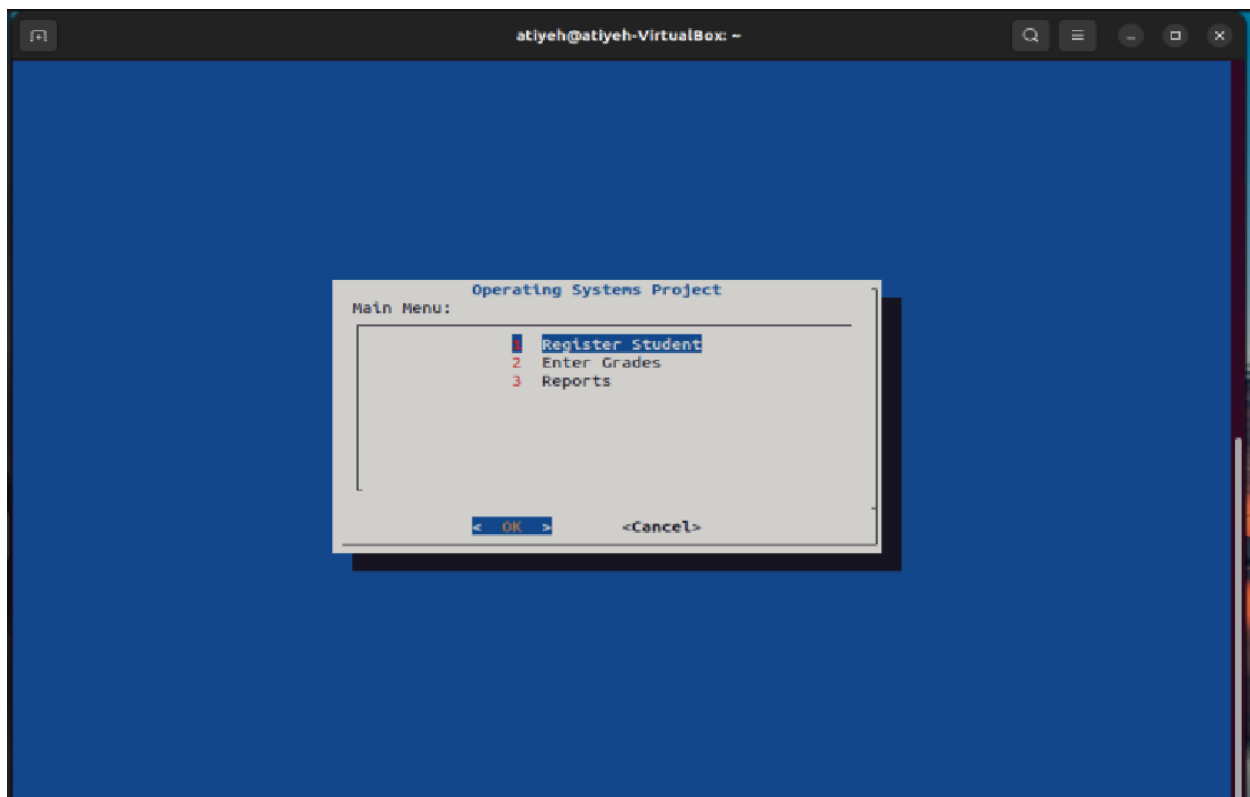
پایان حلقه و تابع

```
done  
}
```

حلقه ادامه دارد تا زمانی که کاربر با Cancel/ESC از منوی اصلی خارج شود.

جمع‌بندی این بخش

در تابع `main_menu`، منوی اصلی برنامه با استفاده از `--menu` `dialog` پیاده‌سازی شده است. این منو دسترسی به تمام بخش‌های پروژه را فراهم می‌کند و پس از اتمام هر عملیات دوباره نمایش داده می‌شود. خروج از برنامه نیز با Cancel یا ESC در همین منو انجام می‌گیرد.



4. ساختار کلی برنامه

پس از اجرای برنامه، منوی اصلی نمایش داده می‌شود که شامل سه گزینه است:

1. Register Student

2. Enter Grades

3. Reports

کاربر می‌تواند با انتخاب هر گزینه وارد بخش مربوطه شود و پس از پایان هر عملیات، برنامه به منوی اصلی بازمی‌گردد.

5. بخش بررسی اولیه و آماده‌سازی

در ابتدای برنامه، فایل‌های داده تعریف و فایل موقت ایجاد می‌شود:

STU_FILE="./students.db"

GRD_FILE="./grades.db"

TMP=\$(mktemp)

همچنین بررسی می‌شود که ابزار dialog نصب باشد و فایل‌های داده قابل ایجاد باشند.

6. ثبت دانشجو (Register Student)

در این بخش، اطلاعات زیر از کاربر دریافت می‌شود:

• Student ID

• First Name

• Last Name

```

register_students() {
    while true; do
        dialog --clear --title "Student Registration" \
            --form "Enter student information:" 12 60 0 \
            "Student ID:" 1 1 "" 1 18 30 0 \
            "First Name:" 2 1 "" 2 18 30 0 \
            "Last Name:" 3 1 "" 3 18 30 0 \
            2> "$TMP"

        # Cancel/ESC -> back to main menu
        [ $? -ne 0 ] && return

        id=$(sed -n '1p' "$TMP" | tr -d ' ')
        fn=$(sed -n '2p' "$TMP")
        ln=$(sed -n '3p' "$TMP")

        if [ -z "$id" ] || [ -z "$fn" ] || [ -z "$ln" ]; then
            msg_error "All fields are required."
            continue
        fi

        if ! is_integer "$id"; then
            msg_error "Student ID must be numeric."
            continue
        fi

        if student_exists "$id"; then
            dialog --yesno "Student ID already exists.\nUpdate existing record?" 8 50
            if [ $? -eq 0 ]; then
                upsert_student "$id" "$fn" "$ln" || msg_error "Update failed."
            else
                continue
            fi
        else
            echo "${id}|${fn}|${ln}" >> "$STU_FILE" || { msg_error "Cannot write to $STU_FILE"; continue; }
        fi

        dialog --yesno "Do you want to register another student?" 7 50
        [ $? -ne 0 ] && return
    done
}

```

توضیح خطبه‌خط تابع register_students

تعریف تابع

```
register_students() {
```

در این خط، تابعی با نام register_students تعریف شده است که وظیفه‌ی ثبت اطلاعات دانشجویان را بر عهده دارد. این تابع از منوی اصلی فراخوانی می‌شود.

ایجاد حلقه‌ی تکرار

```
while true; do
```

برای اینکه امکان ثبت چند دانشجو به صورت پشت سر هم وجود داشته باشد، از یک حلقه‌ی بی‌نهایت while true استفاده شده است. خروج از این حلقه با دستور return انجام می‌شود.

نمایش فرم ثبت دانشجو

```
dialog --clear --title "Student Registration" \
```

- صفحه‌ی قبلی پاک می‌شود(--clear)
- عنوان پنجره‌ی دیالوگ برابر با *Student Registration* قرار داده می‌شود

```
--form "Enter student information:" 12 60 0 \
```

- یک فرم دیالوگی ایجاد می‌شود
- متن راهنما *: Enter student information*
- اندازه پنجره: ارتفاع 12 و عرض 60

```
"Student ID:" 1 1 "" 1 18 30 0 \  
"First Name:" 2 1 "" 2 18 30 0 \  
"Last Name:" 3 1 "" 3 18 30 0 \
```

در این قسمت سه فیلد ورودی تعریف شده است:

1. شماره دانشجویی
2. نام
3. نام خانوادگی

هر فیلد شامل موقعیت نمایش و طول ورودی است.

```
2> "$TMP"
```

خروجی فرم (اطلاعات وارد شده توسط کاربر) در فایل موقت \$TMP ذخیره می‌شود.

بررسی لغو عملیات

```
[ $? -ne 0 ] && return
```

اگر کاربر دکمه **Cancel** یا **ESC** را فشار دهد:

- مقدار خروجی `dialog` غیر صفر می‌شود
- تابع خاتمه یافته و کنترل به منوی اصلی باز می‌گردد

استخراج مقادیر وارد شده

```
id=$(sed -n '1p' "$TMP" | tr -d ' ')  
fn=$(sed -n '2p' "$TMP")  
ln=$(sed -n '3p' "$TMP")
```

- خط اول فایل موقت → شماره دانشجویی
- خط دوم → نام
- خط سوم → نام خانوادگی
- فضاهای اضافی از شماره دانشجویی حذف می‌شوند.

بررسی خالی نبودن فیلدها

```
if [ -z "$id" ] || [ -z "$fn" ] || [ -z "$ln" ]; then
```

اگر هر یک از فیلدها خالی باشد:

```
msg_error "All fields are required."
continue
```

- پیام خطا نمایش داده می‌شود
- برنامه دوباره به فرم ثبت دانشجو باز می‌گردد

بررسی عددی بودن شماره دانشجویی

```
if ! is_integer "$id"; then
```

در این مرحله بررسی می‌شود که شماره دانشجویی فقط شامل عدد باشد.

```
msg_error "Student ID must be numeric."
continue
```

در صورت نامعتبر بودن، پیام خطا نمایش داده شده و فرم دوباره باز می‌شود.

بررسی تکراری بودن شماره دانشجویی

```
if student_exists "$id"; then
```

اگر شماره دانشجویی قبلاً در فایل دانشجویان وجود داشته باشد:

```
dialog --yesno "Student ID already exists.\nUpdate existing record?" 8 50
```

از کاربر پرسیده می‌شود که آیا مایل به به‌روزرسانی اطلاعات قبلی است یا خیر.

اگر کاربر Yes را انتخاب کند:

```
upsert_student "$id" "$fn" "$ln" || msg_error "Update failed."
```

اطلاعات دانشجو به روز رسانی می‌شود.

اگر **No** را انتخاب کند:

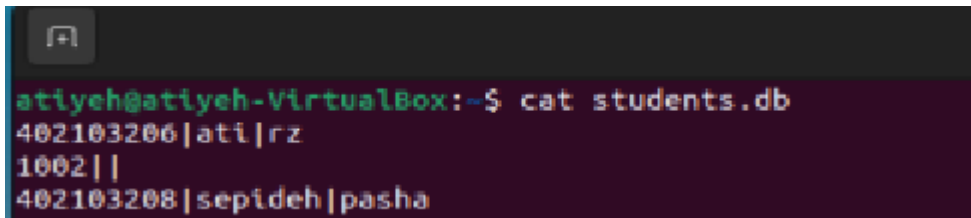
```
continue
```

به فرم ثبت دانشجو باز می‌گردد.

ثبت دانشجوی جدید

```
else  
    echo "${id}|${fn}|${ln}" >> "$STU_FILE"
```

در صورتی که شماره دانشجویی تکراری نباشد، اطلاعات دانشجو به فایل `students.db` اضافه می‌شود.



```
atiyeh@atiyeh-VirtualBox:~$ cat students.db  
402103206|ati|rz  
1002||  
402103208|sepideh|pasha
```

```
|| { msg_error "Cannot write to $STU_FILE"; continue; }
```

اگر نوشتن در فایل با خطا مواجه شود، پیام خطا نمایش داده می‌شود.

پرسش برای ادامه ثبت

```
dialog --yesno "Do you want to register another student?" 7 50
```

پس از ثبت دانشجو، از کاربر سؤال می‌شود که آیا مایل به ثبت دانشجوی دیگری است یا خیر.

```
[ $? -ne 0 ] && return
```

- اگر **No** انتخاب شود خروج از تابع و بازگشت به منوی اصلی
- اگر **Yes** ثبت دانشجوی بعدی
-

پایان حلقه و تابع

```
done  
}
```

- حلقه پایان می‌یابد
- تابع `register_students` به‌طور کامل اجرا شده است

جمع‌بندی این بخش

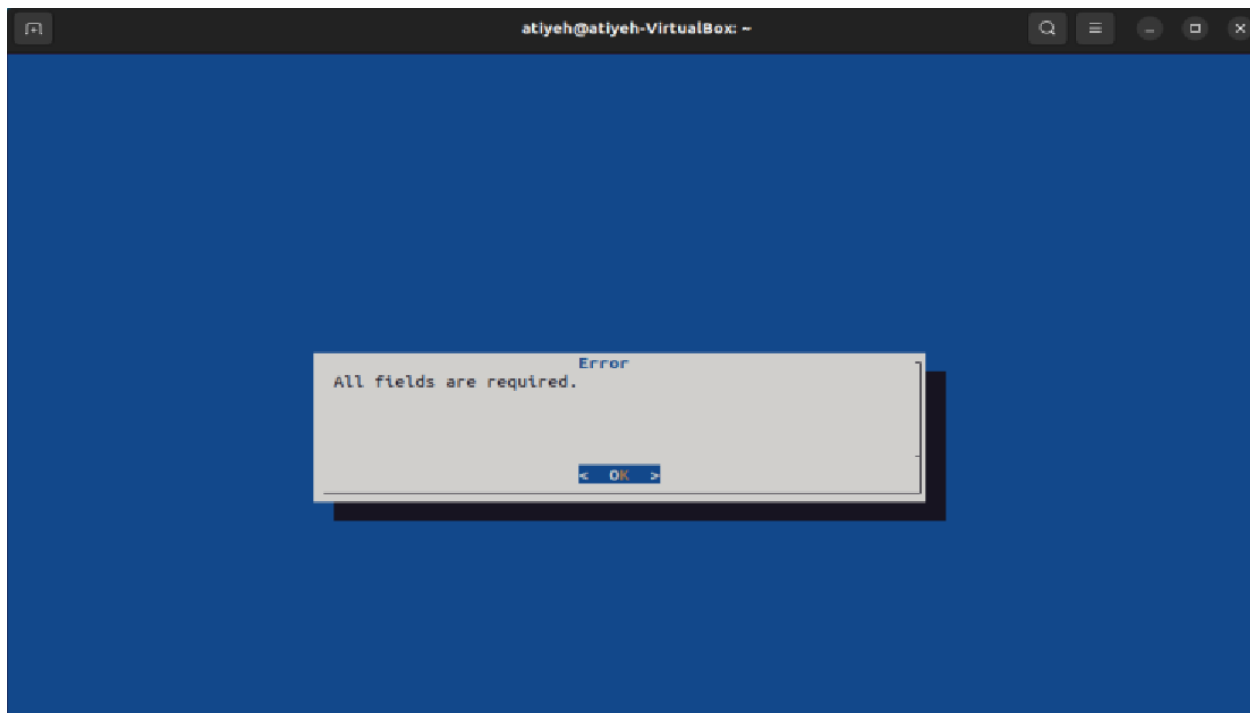
در این تابع، فرآیند ثبت دانشجو به‌صورت تعاملی و ایمن پیاده‌سازی شده است. تمامی خطاهای رایج شامل ورود اطلاعات ناقص، شماره دانشجویی نامعتبر و تکراری مدیریت شده و امکان ثبت چند دانشجو به‌صورت متوالی فراهم شده است

کنترل خطاها:

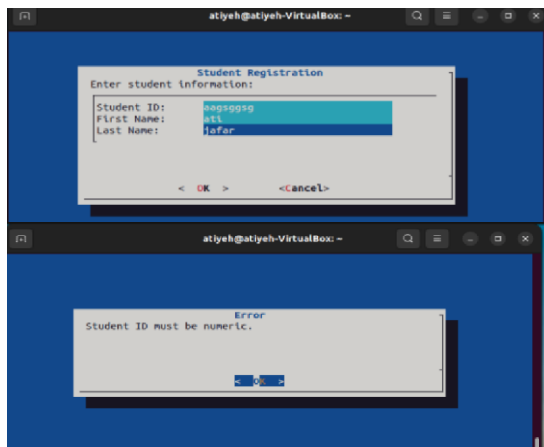
- هیچ فیلدی نباید خالی باشد
- شماره دانشجویی باید عددی باشد
- اگر شماره دانشجویی تکراری باشد، از کاربر سؤال می‌شود که آیا اطلاعات قبلی به‌روزرسانی شود یا خیر

تست‌ها:

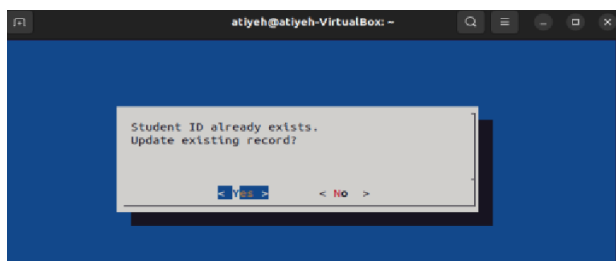
1. وارد کردن فقط ID بدون نام → نمایش پیام خطا



2. وارد کردن ID غیر عددی → پیام خطا



3. وارد کردن ID تکراری → نمایش پیام Update



4. ثبت موفق دانشجو و بازگشت به منوی اصلی

7. ورود نمرات (Enter Grades)

در این بخش، نمرات دانشجو دریافت می‌شود:

- Student ID
- نمره درس سیستم‌عامل
- نمره آزمایشگاه سیستم‌عامل

```
enter_grades() {
while true; do
dialog --clear --title "Enter Grades" \
--form "Enter student ID and grades:" 13 70 0 \
"Student ID:"      1 1 "" 1 30 25 0 \
"Operating Systems Grade:" 2 1 "" 2 30 10 0 \
"OS Lab Grade:"    3 1 "" 3 30 10 0 \
2> "$TMP"

[ $? -ne 0 ] && return

id=$(sed -n '1p' "$TMP" | tr -d ' ')
os=$(sed -n '2p' "$TMP" | tr -d ' ')
lab=$(sed -n '3p' "$TMP" | tr -d ' ')

if [ -z "$id" ] || [ -z "$os" ] || [ -z "$lab" ]; then
msg_error "All fields are required."
continue
fi

if ! is_integer "$id"; then
msg_error "Student ID must be numeric."
continue
fi

if ! student_exists "$id"; then
msg_error "This student is not registered.\nPlease register the student first."
continue
fi

if ! is_valid_grade "$os" || ! is_valid_grade "$lab"; then
msg_error "Grades must be between 0 and 20.\nDecimals allowed (e.g., 15.5)."
continue
fi

if grade_exists "$id"; then
dialog --yesno "Grades already exist for this student.\nUpdate existing grades?" 8 55
if [ $? -eq 0 ]; then
upsert_grade "$id" "$os" "$lab" || msg_error "Update failed."
else
continue
fi
else
echo "${id}|${os}|${lab}" >> "$GRD_FILE" || { msg_error "Cannot write to $GRD_FILE"; continue; }
fi

dialog --yesno "Do you want to enter grades for another student?" 7 60
[ $? -ne 0 ] && return
done
}
```

توضیح خطبه خط تابع enter_grades

تعریف تابع

```
enter_grades() {
```

این تابع برای وارد کردن نمرات دانشجویان طراحی شده است و از منوی اصلی فراخوانی می‌شود.

حلقه برای ورود چند دانشجو پشت سر هم

```
while true; do
```

برای اینکه کاربر بتواند نمرات چند دانشجو را پشت سر هم وارد کند، از حلقه‌ی بی‌نهایت استفاده شده است. خروج از حلقه با `return` انجام می‌شود.

فرم ورود نمرات با `dialog`

```
dialog --clear --title "Enter Grades" \
```

- صفحه قبلی پاک می‌شود
- عنوان پنجره `Enter Grades` :

```
--form "Enter student ID and grades:" 13 70 0 \
```

یک فرم ورودی با متن راهنما ایجاد می‌شود. اندازه پنجره: ارتفاع 13 و عرض 70.

```
"Student ID:" 1 1 "" 1 30 25 0 \  
"Operating Systems Grade:" 2 1 "" 2 30 10 0 \  
"OS Lab Grade:" 3 1 "" 3 30 10 0 \
```

سه ورودی دریافت می‌شود:

1. شماره دانشجویی
2. نمره درس سیستم عامل
3. نمره آزمایشگاه سیستم عامل

```
2> "$TMP"
```

خروجی فرم (اطلاعات وارد شده) در فایل موقت `$TMP` ذخیره می‌شود.

خروج با Cancel/ESC

```
[ $? -ne 0 ] && return
```

اگر کاربر Cancel یا ESC بزند، dialog کد خروجی غیر صفر می‌دهد و تابع تمام شده و برنامه به منوی اصلی بازمی‌گردد.

استخراج داده‌ها از فایل موقت

```
id=$(sed -n '1p' "$TMP" | tr -d ' ')\nos=$(sed -n '2p' "$TMP" | tr -d ' ')\nlab=$(sed -n '3p' "$TMP" | tr -d ' ')
```

- خط اول: شماره دانشجویی
 - خط دوم: نمره سیستم عامل
 - خط سوم: نمره آزمایشگاه
- همچنین فاصله‌های اضافی حذف می‌شوند تا ورودی تمیز ذخیره شود.

بررسی خالی نبودن فیلدها

```
if [ -z "$id" ] || [ -z "$os" ] || [ -z "$lab" ]; then\n    msg_error "All fields are required."\n    continue\nfi
```

اگر هر کدام از فیلدها خالی باشد:

- پیام خطا نمایش داده می‌شود
- با continue دوباره فرم باز می‌شود

بررسی عددی بودن شماره دانشجویی

```
if ! is_integer "$id"; then\n    msg_error "Student ID must be numeric."\n    continue\nfi
```

در این قسمت بررسی می‌شود Student ID عددی باشد. در غیر این صورت خطا داده شده و فرم دوباره نمایش داده می‌شود.

بررسی ثبت بودن دانشجو

```
if ! student_exists "$id"; then
    msg_error "This student is not registered.\nPlease register the student
first."
    continue
fi
```

اگر شماره دانشجویی قبلاً در فایل دانشجویان ثبت نشده باشد:

- اجازه ثبت نمره داده نمی‌شود
- پیام خطا نمایش داده می‌شود
- فرم دوباره باز می‌شود

بررسی معتبر بودن نمره‌ها

```
if ! is_valid_grade "$os" || ! is_valid_grade "$lab"; then
    msg_error "Grades must be between 0 and 20.\nDecimals allowed (e.g.,
15.5)."
```

```
    continue
fi
```

نمره‌ها باید:

- عددی باشند
- بین 0 تا 20 باشند
- امکان اعشاری نیز وجود دارد (مثلاً 15.5)

اگر نمره نامعتبر باشد، پیام خطا نمایش داده می‌شود و فرم دوباره اجرا می‌شود.

بررسی اینکه آیا قبلاً نمره ثبت شده یا نه (Update)

```
if grade_exists "$id"; then
```

اگر برای این دانشجو قبلاً نمره وجود داشته باشد:

```
dialog --yesno "Grades already exist for this student.\nUpdate existing
grades?" 8 55
```

از کاربر پرسیده می‌شود که آیا مایل به به‌روزرسانی نمرات قبلی است یا خیر.

اگر: Yes

```
upsert_grade "$id" "$os" "$lab" || msg_error "Update failed."
```

نمرات دانشجو در فایل بهروزرسانی می‌شود.

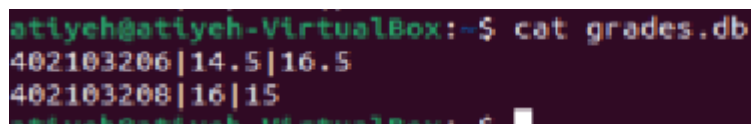
اگر: No

```
continue
```

به فرم برمی‌گردد تا کاربر داده‌ی دیگری وارد کند.

ثبت نمرات جدید در فایل

```
else
    echo "${id}|${os}|${lab}" >> "$GRD_FILE" || { msg_error "Cannot write to
$GRD_FILE"; continue; }
fi
```



```
otiyeh@otiyeh-VirtualBox:~$ cat grades.db
402103206|14.5|16.5
402103208|16|15
otiyeh@otiyeh-VirtualBox:~$
```

اگر نمره قبلاً ثبت نشده باشد، یک رکورد جدید به فایل نمرات اضافه می‌شود.
در صورت خطا در نوشتن فایل، پیام مناسب نمایش داده می‌شود.

پرسش ادامه ورود نمره

```
dialog --yesno "Do you want to enter grades for another student?" 7 60
[ $? -ne 0 ] && return
```

پس از ثبت نمره، از کاربر سؤال می‌شود آیا مایل است نمرات دانشجوی دیگری را وارد کند یا خیر.
اگر No انتخاب شود، تابع پایان یافته و به منوی اصلی بازمی‌گردد.

پایان حلقه و تابع

```
done
}
```

حلقه و تابع تمام می‌شود.

جمع‌بندی این بخش

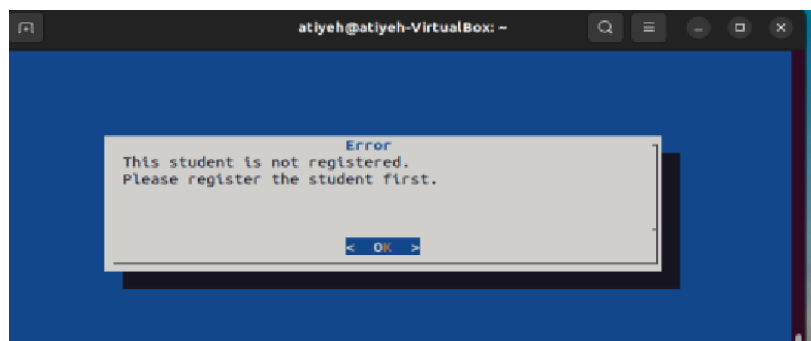
در این تابع، ورود نمرات به‌صورت دیالوگی پیاده‌سازی شده و خطاهای رایج شامل ورودی ناقص، شماره دانشجویی نامعتبر، ثبت نبودن دانشجو، نمره نامعتبر و نمره تکراری مدیریت شده است. همچنین امکان ورود چندین نمره پشت سر هم و بازگشت صحیح به منوی اصلی فراهم گردیده است.

کنترل خطاها:

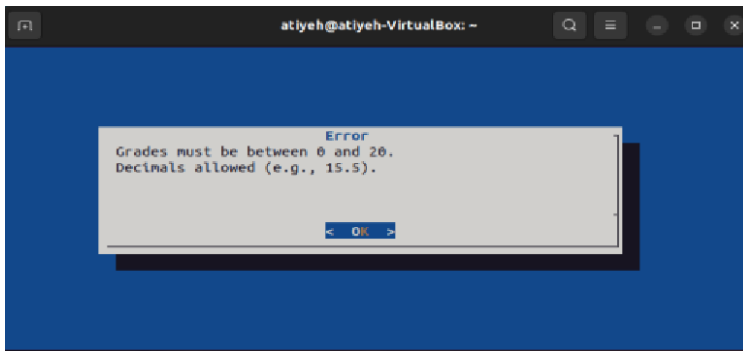
- هیچ فیلدی نباید خالی باشد
- شماره دانشجویی باید عددی باشد
- دانشجو باید قبلاً ثبت شده باشد
- نمرات باید عددی و در بازه 0 تا 20 باشند
- در صورت ثبت قبلی نمرات، امکان Update وجود دارد

تست‌ها:

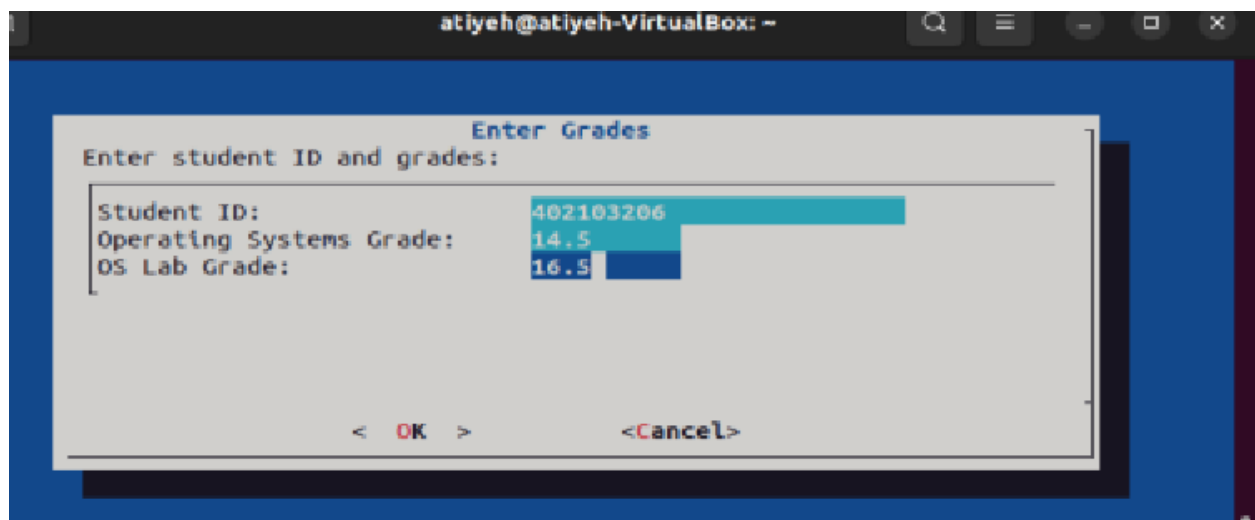
1. وارد کردن نمره برای دانشجوی ثبت‌نشده → پیام خطا



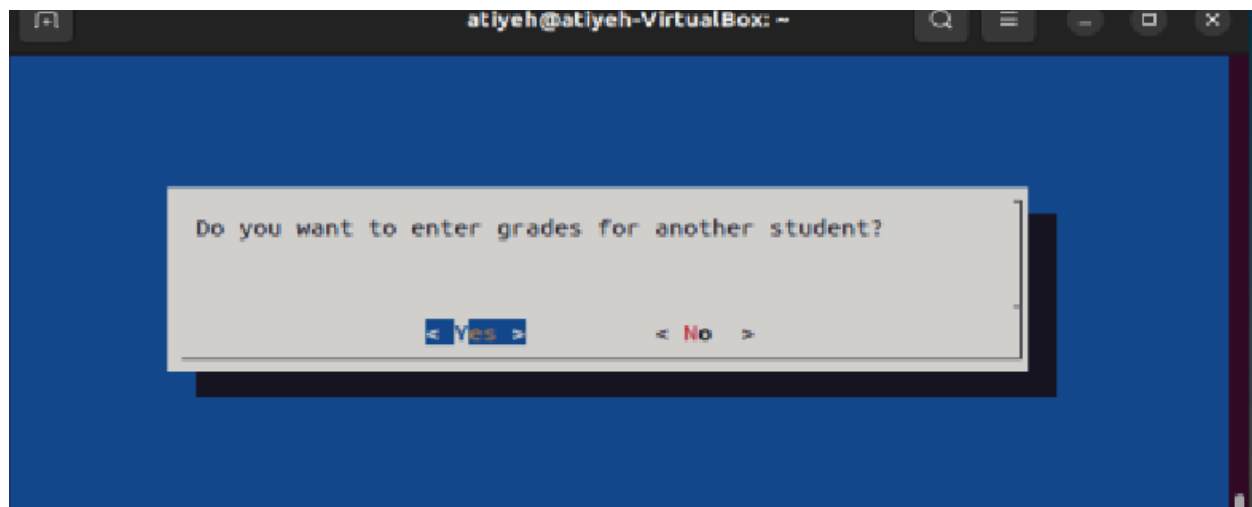
2. وارد کردن نمره خارج از بازه → پیام خطا



3. ثبت نمره صحیح

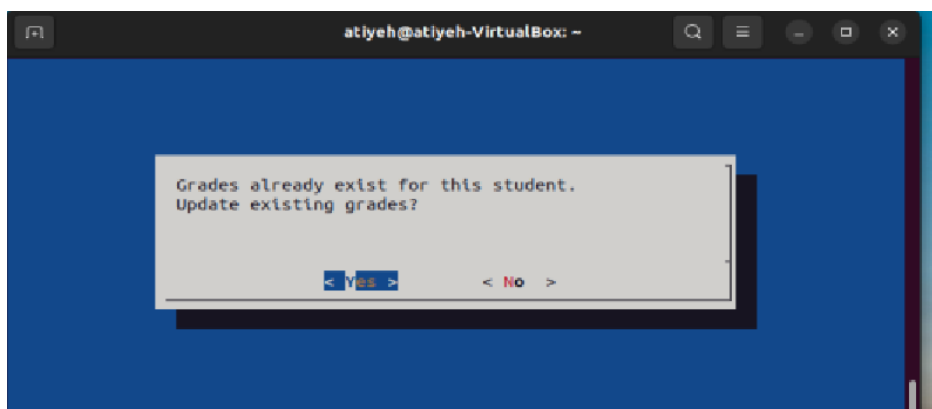


بد از زدن دکمه اوکی منوی به این مبنا باز میشه



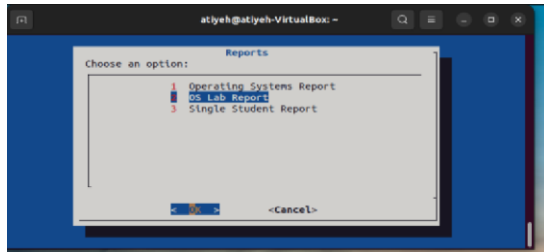
که با زدن دکمه no به منوی اصلی و با زدن دکمه yes به قسمت ثبت نمرات مجدد وارد میشیم

4. Update نمره برای دانشجوی دارای نمره



8. گزارش‌گیری (Reports)

این بخش شامل سه نوع گزارش است :

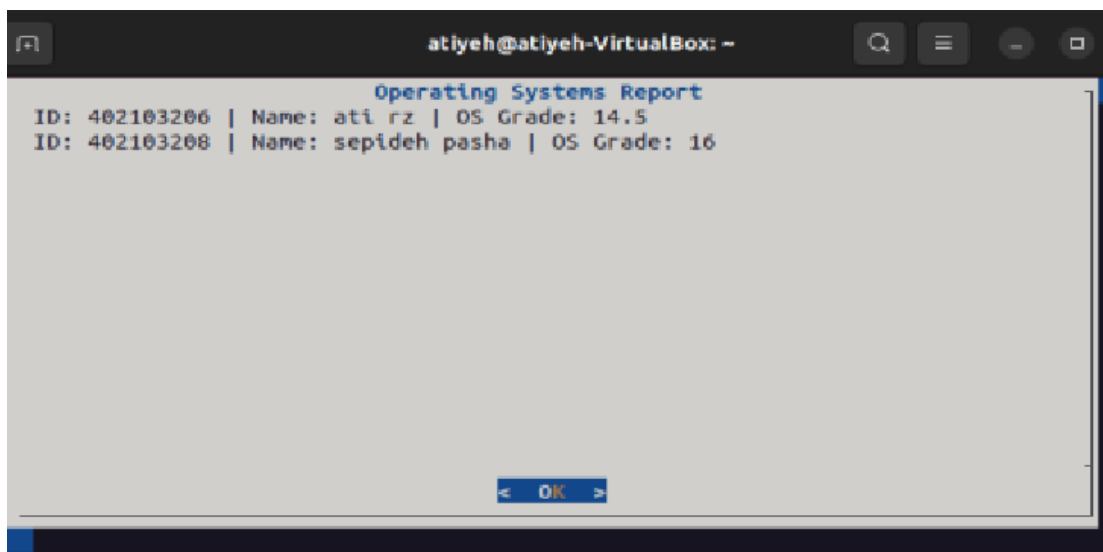


گزارش درس سیستم‌عامل

نمایش:

- Student ID
- Name
- Operating Systems Grade

```
report_os() {  
    if [ ! -s "$GRD_FILE" ]; then  
        dialog --title "Operating Systems Report" --msgbox "No grades data available." 7 45  
        return  
    fi  
  
    output=$(  
        awk -F'|' 'NR==FNR { s[$1]=$2 " $3; next }  
        { name=s[$1]; if(name=="") name="Unknown Student";  
        print "ID: "$1" | Name: "name" | OS Grade: "$2  
        }'  
        "$STU_FILE" "$GRD_FILE"  
    )  
  
    [ -z "$output" ] && output="No data available."  
    dialog --title "Operating Systems Report" --msgbox "$output" 20 80  
}
```



report_os توضیح خطہ خط تابع

تعريف تابع

```
report_os() {
```

این تابع مسئول گزارش‌گیری نمرات درس سیستم عامل برای تمام دانشجویان ثبت شده است. این تابع از منوی Reports فراخوانی می‌شود.

بررسی وجود داده نمرات

```
if [ ! -s "$GRD FILE" ]; then
```

در این خط بررسی می‌شود که:

- فایل نمرات (grades.db)
- یا وجود ندارد
- یا خالی است (-s) یعنی فایل غیر خالی باشد

```
dialog --title "Operating Systems Report" --msgbox "No grades data
available." 7 45
return
```

اگر فایل نمرات خالی باشد:

- یک پیام مناسب به کاربر نمایش داده می‌شود
- تابع خاتمه می‌یابد و به منوی قبلی باز می‌گردد

تولید خروجی گزارش باawk

```
output=$ (
```

در این قسمت خروجی گزارش در یک متغیر به نام `output` ذخیره می‌شود تا بعداً در `dialog` نمایش داده شود.

خواندن اطلاعات دانشجویان و نمرات

```
awk -F'|' '{
```

- دستور `awk` برای پردازش فایل‌ها استفاده شده است
- جداکننده فیلدها علامت `|` می‌باشد

پردازش فایل students.db

```
NR==FNR { s[$1]=$2 " "$3; next }
```

این بخش فقط هنگام خواندن فایل اول (**students.db**) اجرا می‌شود:

- شماره دانشجویی (\$1) به عنوان کلید
- نام و نام خانوادگی (\$2 \$3) به عنوان مقدار
- همهی این‌ها در یک آرایه به نام s ذخیره می‌شوند

یعنی:

```
s[StudentID] = "FirstName LastName"
```

پردازش فایل grades.db

```
{
  name=s[$1];
  if(name=="") name="Unknown Student";
  print "ID: "$1" | Name: "name" | OS Grade: "$2
}
```

در این بخش که مربوط به فایل نمرات است:

- با استفاده از Student ID ، نام دانشجو از آرایه پیدا می‌شود
- اگر نامی پیدا نشود، مقدار **Unknown Student** نمایش داده می‌شود
- سپس اطلاعات زیر چاپ می‌شود:
 - Student ID
 - نام دانشجو
 - نمره درس سیستم عامل

فقط نمره درس **Operating Systems** نمایش داده می‌شود (مطابق صورت سؤال).

مشخص کردن ترتیب فایل‌ها

```
' "$STU_FILE" "$GRD_FILE"
```

ابتدا فایل دانشجویان خوانده می‌شود و سپس فایل نمرات، تا ارتباط بین آن‌ها برقرار شود.

بررسی خالی نبودن خروجی

```
[ -z "$output" ] && output="No data available."
```

اگر به هر دلیلی خروجی تولید نشده باشد، یک پیام جایگزین در نظر گرفته می‌شود تا `dialog` خالی نمایش داده نشود.

نمایش گزارش در `dialog`

```
dialog --title "Operating Systems Report" --msgbox "$output" 20 80
```

در نهایت، گزارش تولیدشده در یک پنجره‌ی دیالوگی با عنوان **Operating Systems Report** نمایش داده می‌شود.

پایان تابع

```
}
```

جمع‌بندی این بخش

در تابع `report_os`، گزارش نمرات درس سیستم‌عامل با استفاده از پردازش فایل‌ها و دستور `awk` پیاده‌سازی شده است. این گزارش اطلاعات دانشجویان و نمرات آن‌ها را ترکیب کرده و با مدیریت خطاهای احتمالی، خروجی مناسب را در قالب پنجره‌ی دیالوگی نمایش می‌دهد.

گزارش آزمایشگاه سیستم عامل

نمایش:

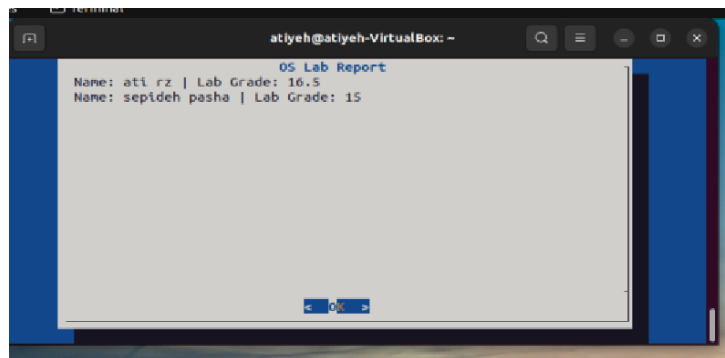
- Name
- OS Lab Grade

(شماره دانشجویی نمایش داده نمی‌شود)

```
report_lab() {
    if [ ! -s "$GRD_FILE" ]; then
        dialog --title "OS Lab Report" --msgbox "No grades data available." 7 45
        return
    fi

    output=$(
        awk -F'|' '
        NR==FNR { s[$1]=$2 " "$3; next }
        { name=s[$1]; if(name=="") name="Unknown Student";
          print "Name: "name | Lab Grade: "$3
        }
        ' "$STU_FILE" "$GRD_FILE"
    )

    [ -z "$output" ] && output="No data available."
    dialog --title "OS Lab Report" --msgbox "$output" 20 70
}
```



توضیح خطبه خط تابع report_lab

تعریف تابع

```
report_lab() {
```

این تابع مسئول گزارش‌گیری نمرات آزمایشگاه سیستم عامل برای تمام دانشجویان است و از منوی Reports فراخوانی می‌شود.

بررسی وجود داده نمرات

```
if [ ! -s "$GRD_FILE" ]; then
```

ابتدا بررسی می‌شود فایل نمرات (grades.db) وجود داشته و خالی نباشد.

```
dialog --title "OS Lab Report" --msgbox "No grades data available." 7 45  
return
```

اگر فایل خالی باشد:

- پیام مناسب نمایش داده می‌شود
- تابع خاتمه می‌یابد و به منوی گزارش‌گیری باز می‌گردد

«No grades data available» در گزارش آزمایشگاه وقتی نمره‌ای ثبت نشده

تولید خروجی گزارش باawk

```
output=$(
```

خروجی تولیدشده توسط awk در متغیر output ذخیره می‌شود تا در dialog نمایش داده شود.

اجرای awk با جداکننده |

```
awk -F'|' ' '
```

چون فرمت فایل‌ها به شکل field1|field2|... است، جداکننده | در awk تعریف شده است.

خواندن و ذخیره اطلاعات دانشجویان

```
NR==FNR { s[$1]=$2" "$3; next }
```

هنگام خواندن فایل اول: (students.db)

- آرایه‌ای ساخته می‌شود که در آن:
 - کلید = شماره دانشجویی
 - مقدار = نام + نام خانوادگی
- یعنی: `s[ID] = "FirstName LastName"`

پردازش فایل نمرات و تولید گزارش آزمایشگاه

```
{
name=s[$1];
if (name=="") name="Unknown Student";
print "Name: "name" | Lab Grade: "$3
}
```

هنگام خواندن فایل نمرات: (grades.db)

- نام دانشجو بر اساس Student ID پیدا می‌شود
- اگر پیدا نشود، مقدار Unknown Student قرار داده می‌شود
- سپس فقط دو مورد چاپ می‌شود:
 - Name
 - Lab Grade

در این گزارش Student ID چاپ نمی‌شود و تنها نام و نمره آزمایشگاه نمایش داده می‌شود.

ترتیب فایل‌ها

```
' "$STU_FILE" "$GRD_FILE"
```

مثل گزارش قبلی ابتدا فایل دانشجو‌ها و سپس فایل نمرات داده می‌شود تا اطلاعات به هم وصل شوند.

جلوگیری از خروجی خالی

```
[ -z "$output" ] && output="No data available."
```

اگر خروجی تولید نشده باشد، متن جایگزین قرار داده می‌شود.

نمایش گزارش در dialog

```
dialog --title "OS Lab Report" --msgbox "$output" 20 70
```

در پایان، گزارش در یک پنجره‌ی دیالوگی نمایش داده می‌شود.

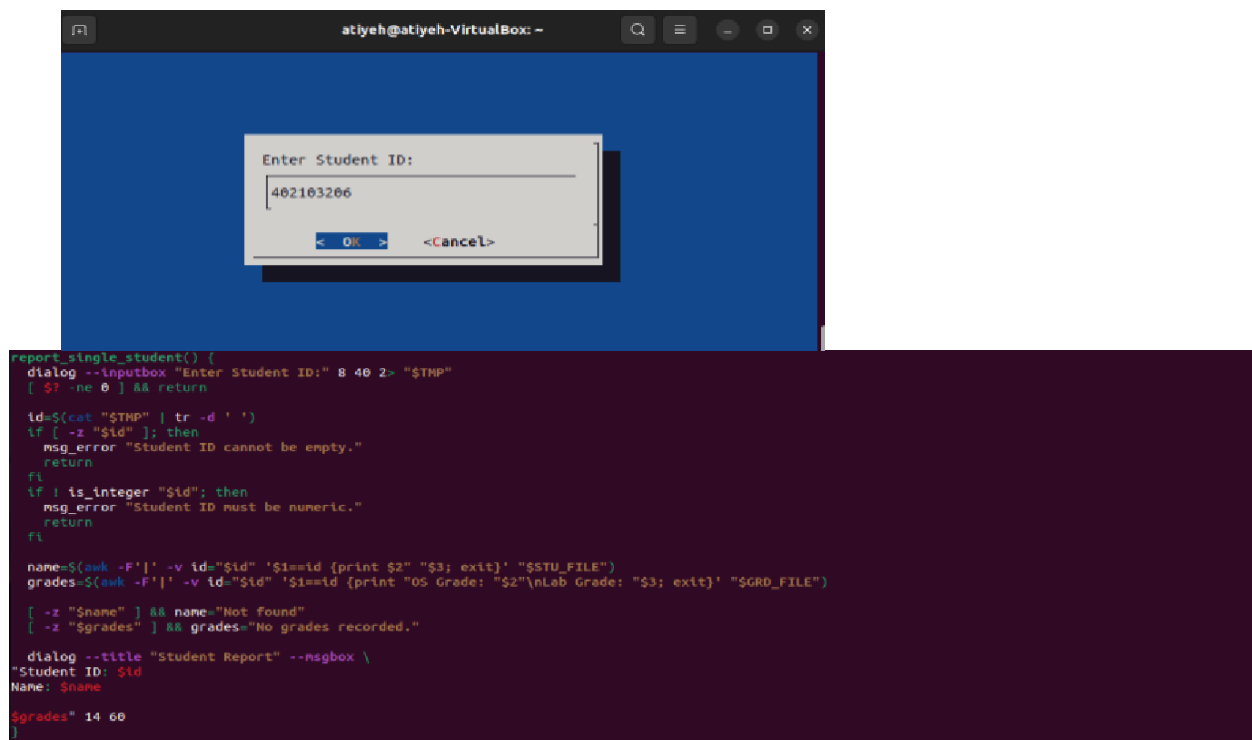
پایان تابع

```
}
```

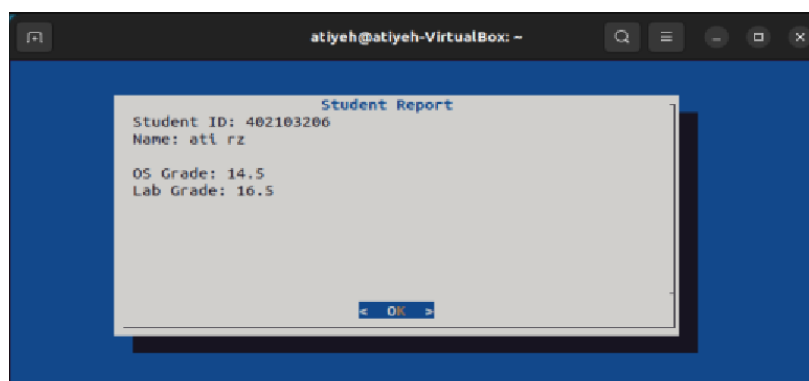
گزارش یک دانشجو

با دریافت شماره دانشجویی:

- نام و نام خانوادگی
- نمرات هر دو درس



```
report_single_student() {  
  dialog --inputbox "Enter Student ID:" 8 40 2> "$TMP"  
  [ $? -ne 0 ] && return  
  
  id=$(cat "$TMP" | tr -d ' ' )  
  if [ -z "$id" ]; then  
    msg_error "Student ID cannot be empty."  
    return  
  fi  
  if ! is_integer "$id"; then  
    msg_error "Student ID must be numeric."  
    return  
  fi  
  
  name=$(awk -F'|' -v id="$id" ' $1==id {print $2" "$3; exit}' "$STU_FILE")  
  grades=$(awk -F'|' -v id="$id" ' $1==id {print "OS Grade: "$2"\nLab Grade: "$3; exit}' "$GRD_FILE")  
  
  [ -z "$name" ] && name="Not found"  
  [ -z "$grades" ] && grades="No grades recorded."  
  
  dialog --title "Student Report" --msgbox \  
"Student ID: $id  
Name: $name  
  
OS Grade: 14.5  
Lab Grade: 16.5  
$grades" 14 60  
}
```



توضیح خطبه خط تابع `report_single_student`

تعریف تابع

```
report_single_student() {
```

این تابع برای گزارش گیری یک دانشجوی خاص استفاده می شود. کاربر شماره دانشجویی را وارد می کند و سپس نام و نمرات همان دانشجو نمایش داده می شود.

دریافت شماره دانشجویی با `inputbox`

```
dialog --inputbox "Enter Student ID:" 8 40 2> "$TMP"
```

یک پنجره ی ورودی (InputBox) نمایش داده می شود تا کاربر **Student ID** را وارد کند. خروجی در فایل موقت `$TMP` ذخیره می شود.

خروج با `Cancel/ESC`

```
[ $? -ne 0 ] && return
```

اگر کاربر `Cancel` یا `ESC` بزند، تابع متوقف می شود و به منوی `Reports` برمی گردد.

خواندن مقدار وارد شده و حذف فاصله ها

```
id=$(cat "$TMP" | tr -d ' ')
```

مقدار وارد شده از فایل موقت خوانده می شود و فاصله های اضافی حذف می گردد تا `Student ID` تمیز پردازش شود.

بررسی خالی نبودن `Student ID`

```
if [ -z "$id" ]; then
```

```
msg_error "Student ID cannot be empty."
return
fi
```

اگر کاربر چیزی وارد نکرده باشد:

- پیام خطا نمایش داده می‌شود
- تابع پایان می‌یابد

بررسی عددی بودن Student ID

```
if ! is_integer "$id"; then
    msg_error "Student ID must be numeric."
    return
fi
```

اگر Student ID عددی نباشد:

- پیام خطا نمایش داده می‌شود
- تابع پایان می‌یابد

پیدا کردن نام دانشجو از فایل دانشجویان

```
name=$(awk -F'|' -v id="$id" '$1==id {print $2" "$3; exit}' "$STU_FILE")
```

:| و جداکننده awk با دستور

- در فایل students.db سطر مربوط به Student ID پیدا می‌شود
- نام و نام خانوادگی چاپ می‌شود
- exit باعث می‌شود بعد از پیدا کردن اولین رکورد، جستجو متوقف شود (سریع‌تر و بهینه‌تر)

پیدا کردن نمرات دانشجو از فایل نمرات

```
grades=$(awk -F'|' -v id="$id" '$1==id {print "OS Grade: "$2"\nLab Grade: "$3; exit}' "$GRD_FILE")
```

در فایل grades.db سطر مربوط به Student ID پیدا می‌شود و خروجی به شکل دو خط تولید می‌شود:

- OS Grade
- Lab Grade

مقداردهی جایگزین در صورت نبودن اطلاعات

```
[ -z "$name" ] && name="Not found"
[ -z "$grades" ] && grades="No grades recorded."
```

اگر Student ID در فایل دانشجویان پیدا نشود:

- نام برابر Not found می‌شود

اگر نمره‌ای برای این Student ID ثبت نشده باشد:

- متن No grades recorded. نمایش داده می‌شود

نمایش گزارش نهایی در msgbox

```
dialog --title "Student Report" --msgbox \
"Student ID: $id
Name: $name

$grades" 14 60
```

در پایان گزارش در یک پنجره‌ی دیالوگی نمایش داده می‌شود شامل:

- Student ID
- Name
- OS Grade
- Lab Grade (اگر موجود باشد)

پایان تابع

```
}
```


تابع reports_menu

```
reports_menu() {
  while true; do
    dialog --clear --title "Reports" \
      --menu "Choose an option:" 13 60 0 \
        1 "Operating Systems Report" \
        2 "OS Lab Report" \
        3 "Single Student Report" \
        2> "$TMP"

    [ $? -ne 0 ] && return

    case "$(cat "$TMP")" in
      1) report_os ;;
      2) report_lab ;;
      3) report_single_student ;;
    esac
  done
}
```

reports_menu توضیح خطبه خط تابع

تعریف تابع

```
reports_menu() {
```

این تابع مسئول نمایش منوی گزارش‌گیری است و کاربر از طریق آن می‌تواند نوع گزارش مورد نظر خود را انتخاب کند.

ایجاد حلقه برای ماندن در منوی گزارش‌ها

```
while true; do
```

برای اینکه کاربر بتواند چند گزارش مختلف را پشت سر هم مشاهده کند، از یک حلقه بی‌نهایت استفاده شده است. خروج از این حلقه با `return` انجام می‌شود.

نمایش منوی گزارش‌ها با dialog

```
dialog --clear --title "Reports" \
```

- صفحه‌ی قبلی پاک می‌شود
- عنوان پنجره برابر با **Reports** قرار می‌گیرد

```
--menu "Choose an option:" 13 60 0 \
```

یک منوی دیالوگی ایجاد می‌شود با متن راهنما *Choose an option*
اندازه پنجره: ارتفاع 13 و عرض 60

تعریف گزینه‌های منو

```
1 "Operating Systems Report" \  
2 "OS Lab Report" \  
3 "Single Student Report" \
```

سه گزینه‌ی گزارش در منو تعریف شده است:

1. گزارش درس سیستم‌عامل
2. گزارش آزمایشگاه سیستم‌عامل
3. گزارش یک دانشجوی خاص

ذخیره انتخاب کاربر

```
2> "$TMP"
```

انتخاب کاربر در فایل موقت \$TMP ذخیره می‌شود تا در ادامه پردازش گردد.

خروج با Cancel یا ESC

```
[ $? -ne 0 ] && return
```

اگر کاربر دکمه Cancel یا ESC را فشار دهد:

- منوی گزارش بسته می‌شود
- برنامه به منوی اصلی باز می‌گردد

بررسی انتخاب کاربر با case

```
case "$(cat "$TMP")" in
```

مقدار انتخاب شده توسط کاربر از فایل موقت خوانده می شود و با استفاده از دستور case بررسی می گردد.

اجرای گزارش درس سیستم عامل

```
1) report_os ;;
```

اگر کاربر گزینه 1 را انتخاب کند، تابع report_os اجرا می شود و گزارش نمرات درس سیستم عامل نمایش داده می شود.

اجرای گزارش آزمایشگاه

```
2) report_lab ;;
```

اگر گزینه 2 انتخاب شود، تابع report_lab اجرا شده و گزارش نمرات آزمایشگاه نمایش داده می شود.

اجرای گزارش یک دانشجو

```
3) report_single_student ;;
```

در صورت انتخاب گزینه 3، تابع report_single_student اجرا می شود و گزارش مربوط به یک دانشجو نمایش داده می شود.

پایان case

```
esac
```

دستور case پایان می یابد و کنترل برنامه دوباره به ابتدای حلقه برمی گردد.

پایان حلقه و تابع

```
done  
}
```

پس از بسته شدن هر گزارش، کاربر دوباره به منوی گزارش‌ها بازمی‌گردد تا بتواند گزارش دیگری را انتخاب کند یا از منو خارج شود.

9. نحوه بازگشت و خروج از برنامه

- بستن هر دیالوگ = بازگشت به منوی قبلی
- بستن منوی اصلی = خروج کامل از برنامه

جمع‌بندی و نتیجه‌گیری

و ابزار Shell Script در این پروژه که مربوط به آزمایشگاه درس سیستم‌عامل می‌باشد، یک برنامه‌ی مدیریتی ساده با استفاده از پیاده‌سازی شد. این برنامه امکان ثبت اطلاعات دانشجویان، ورود نمرات و گزارش‌گیری را در محیط سیستم‌عامل `dialog` در `Linux` به صورت کاملاً تعاملی فراهم می‌کند.

مانند دستورات شرطی، حلقه‌ها، توابع، پردازش فایل‌ها و مدیریت `Shell` در فرآیند پیاده‌سازی، سعی شده است از مفاهیم مهم خطا استفاده شود. همچنین با ذخیره‌سازی اطلاعات در فایل، امکان استفاده مجدد از داده‌ها و گزارش‌گیری دقیق فراهم گردیده است.

تمامی بخش‌های برنامه اجرا و تست شده و مطابق صورت‌سؤال پروژه پیاده‌سازی شده‌اند. این پروژه باعث آشنایی عملی با نحوه‌ی کار با دستورات `Shell` و طراحی برنامه‌های مبتنی بر ترمینال در سیستم‌عامل `Linux` شد.